



Sichuan University -
Pittsburgh Institute

ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute

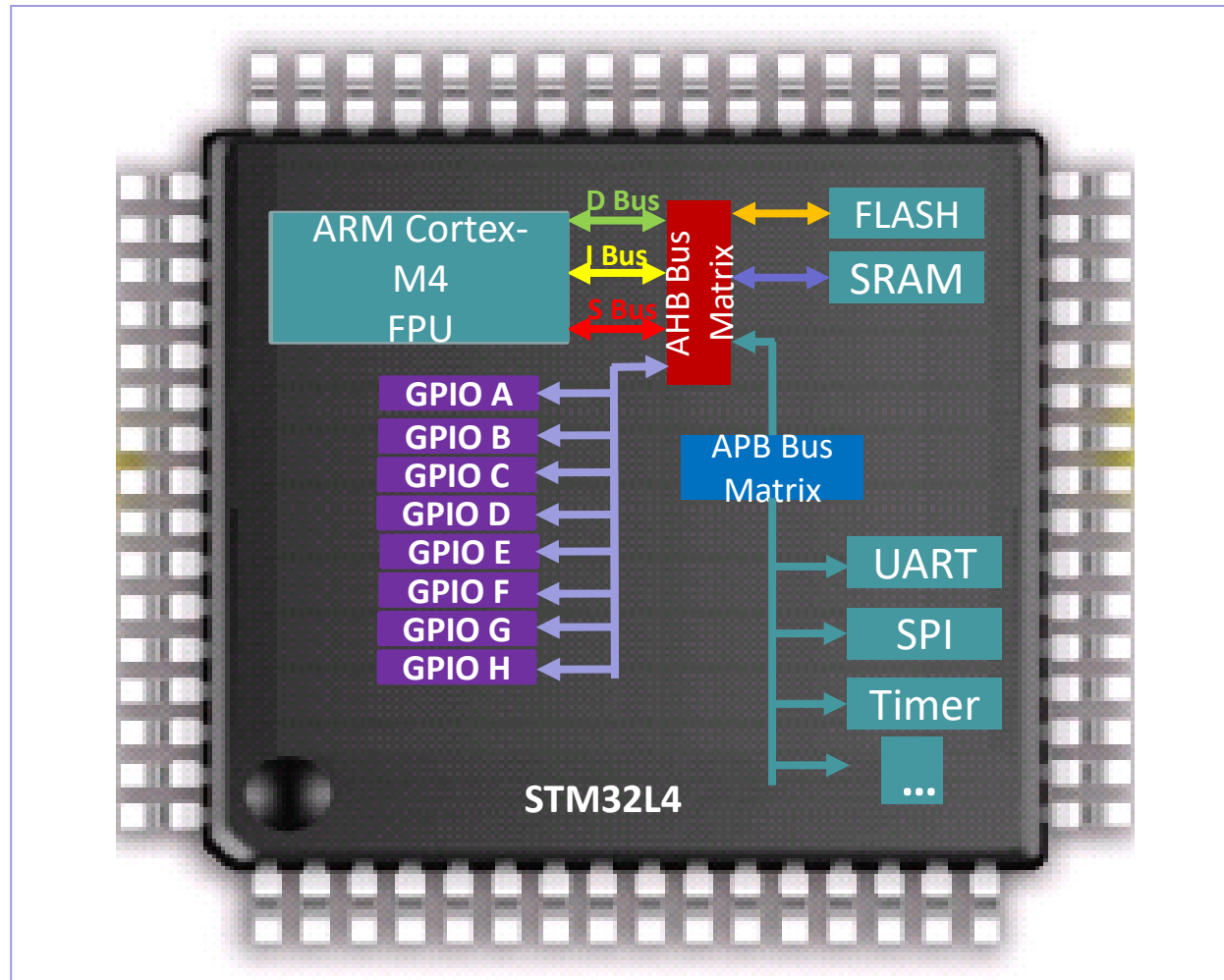


Topics

- Serial Peripheral Interface (SPI)
- How to Program SPI
- Example

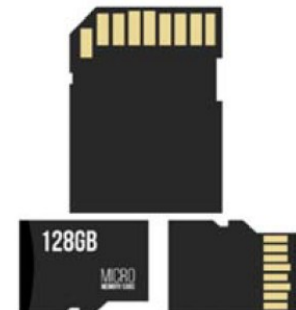
Serial Peripheral Interface (SPI)

3 SPIs:
SPI1, SPI2, SPI3



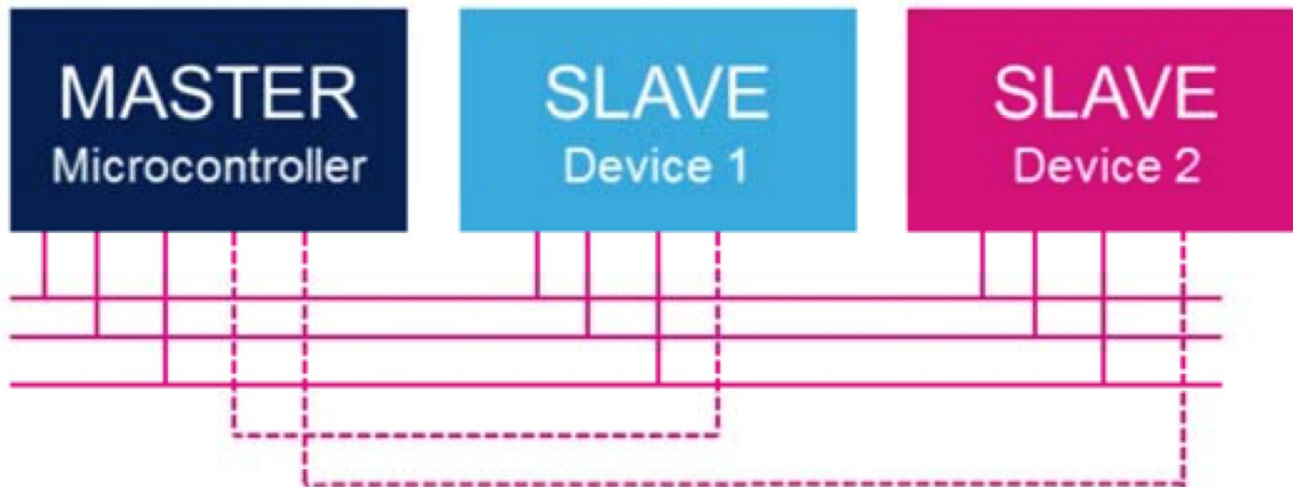
Application Examples

- Displays
- Smart sensors
- Memories
- MMC/SD cards
- IO expanders



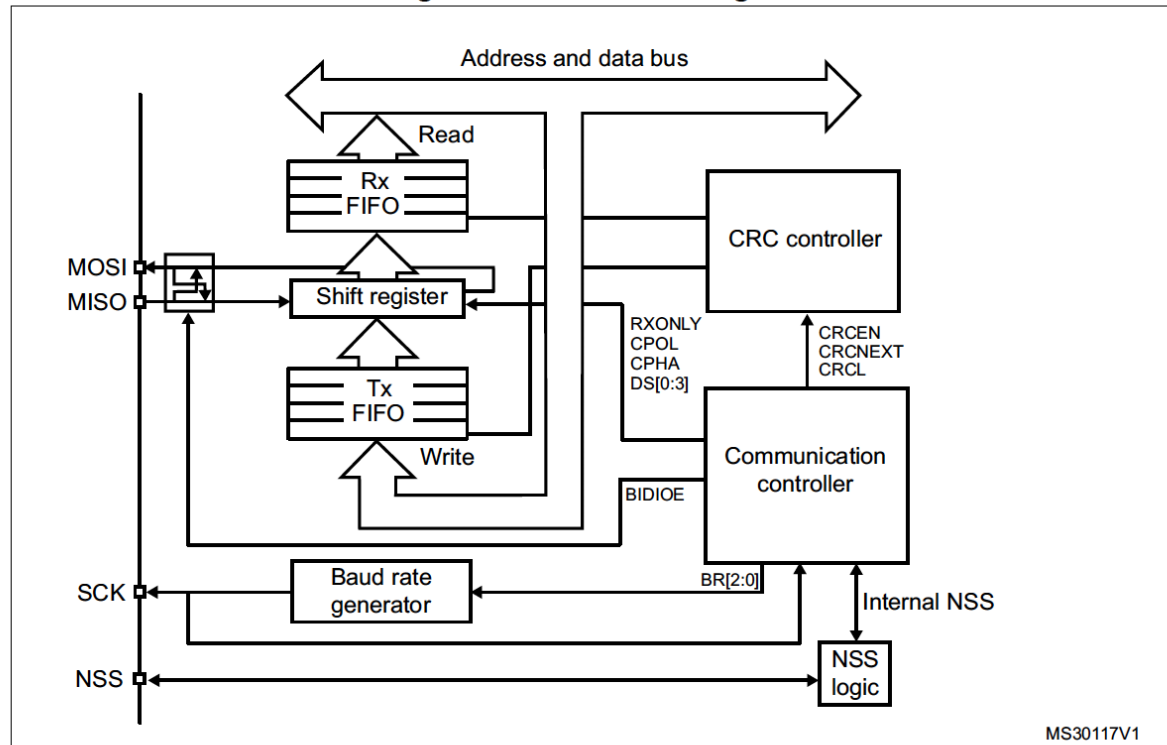
Serial Peripheral Interface (SPI)

- Synchronous communication
- Can have multiple slave devices
- No flow control or acknowledgment
- Slave cannot communicate with slave directly.



Block Diagram

Figure 416. SPI block diagram



MOSI: master out slave in

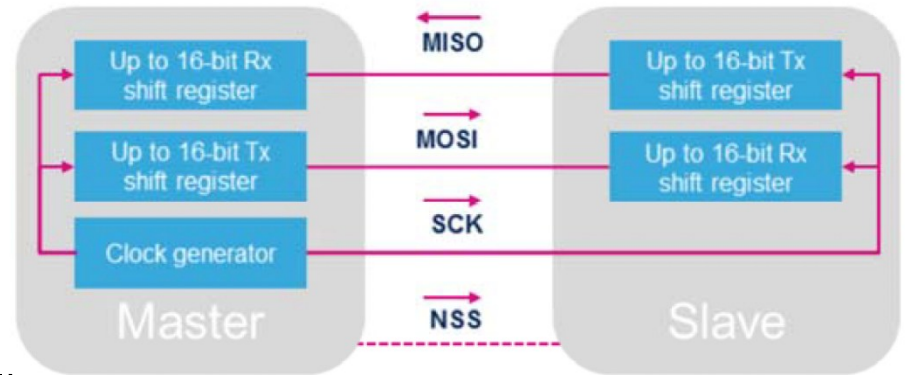
MISO: master in slave out

SCK: serial clock

NSS: slave select (active low)

Interconnection of SPI nodes

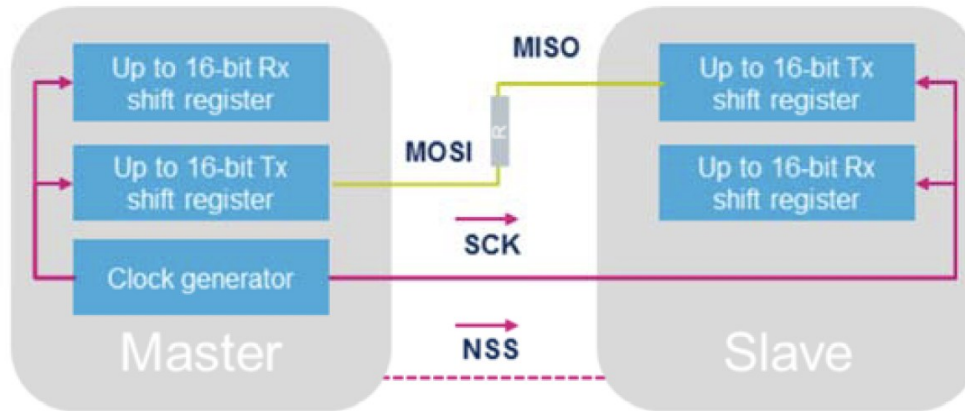
- Master always provides clock
- Data can be exchanged in both directions in parallel
- In **Full-duplex** mode (bidirectional), both master and slave transmit and receive data at the same time
- Full-duplex is the default mode



SPIx_CR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSI	LSB FIRS	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Interconnection of SPI nodes



- In **Half-Duplex** mode (quasi-bidirectional), both master and slave alternate the data transmission and reception synchronously. The nodes share the Single common data line.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 15 **BIDIMODE**: Bidirectional data mode enable. This bit enables half-duplex communication using common single bidirectional data line. **Keep RXONLY bit clear when bidirectional mode is active.**

0: 2-line unidirectional data mode selected
1: 1-line bidirectional data mode selected

Bit 14 **BIDIOE**: Output enable in bidirectional mode

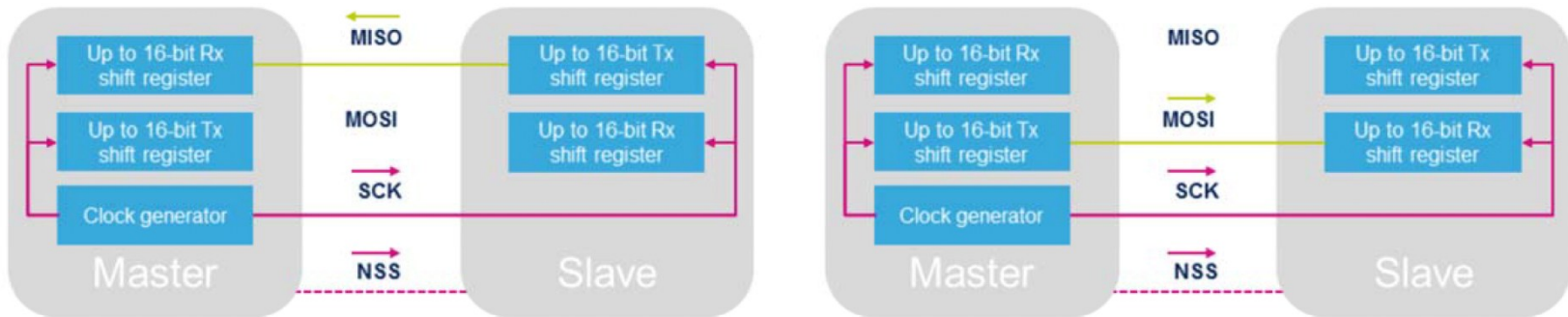
This bit combined with the BIDIMODE bit selects the direction of transfer in bidirectional mode

0: Output disabled (receive-only mode)

1: Output enabled (transmit-only mode)

Note: In master mode, the MOSI pin is used and in slave mode, the MISO pin is used.

Interconnection of SPI nodes



- In **Simplex** mode (unidirectional), both master and slave transmit and receive data at the same time

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRC	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

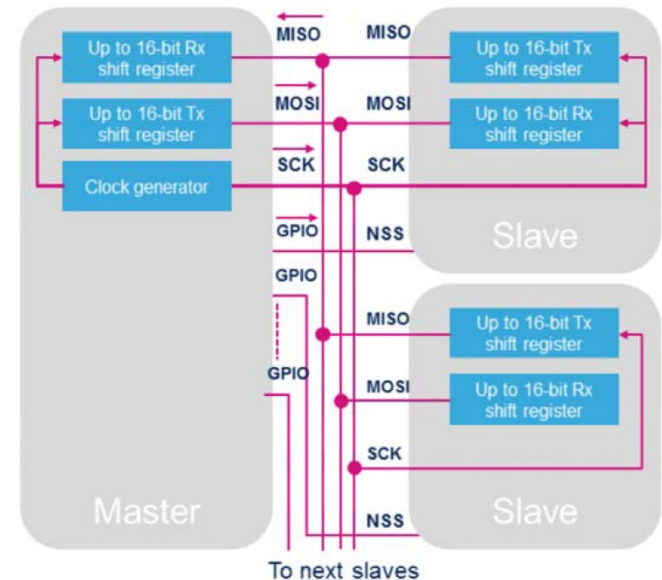
Bit 10 **RXONLY**: Receive only mode enabled.

This bit enables simplex communication using a single unidirectional line to receive data exclusively. Keep BIDIMODE bit clear when receive only mode is active. This bit is also useful in a multislave system in which this particular slave is not accessed, the output from the accessed slave is not corrupted.

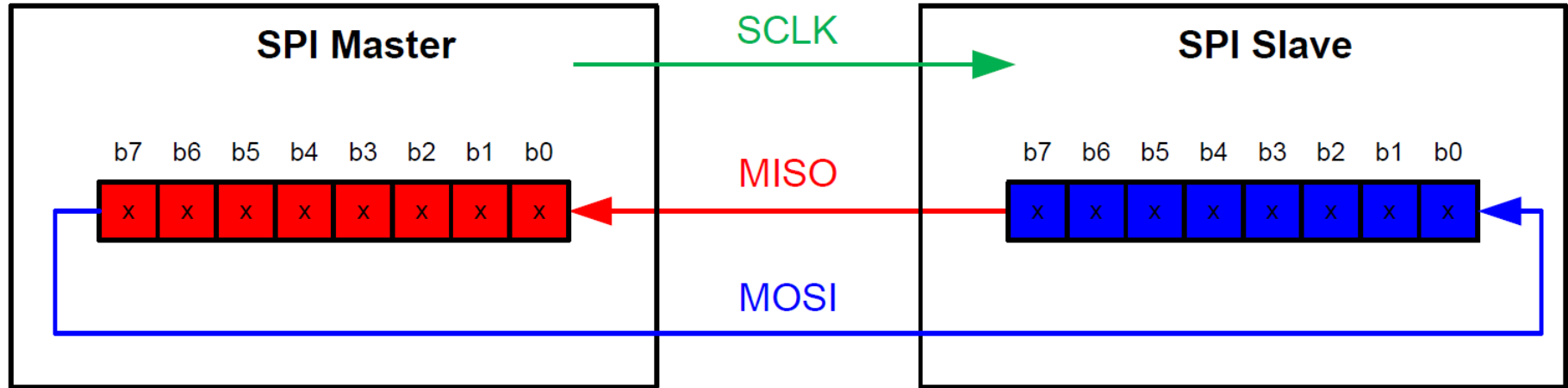
- 0: Full duplex (Transmit and receive)
- 1: Output disabled (Receive-only mode)

Interconnection of SPI nodes

- Multi-slave: Star topology
 - Master selects a single slave node when writing/reading data
 - Separated Slave Select signals (simulated by GPIO pins) are required
 - Slave nodes can have different clock and data formats

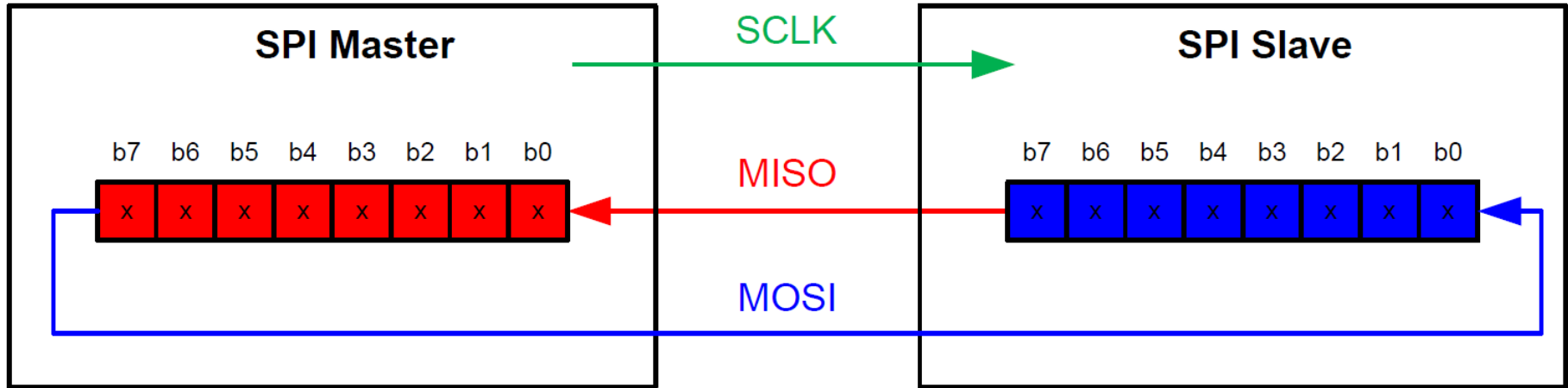


Data Exchange



- Master has to provide clock to slave
- Synchronous exchange: for each clock pulse, a bit is shifted out and another bit is shifted in at the same time. This process stops when all bits are swapped.
- Only master can start the data transfer

Data Exchange



- Size of data frame: from 4 to 16 bits

SPIx_CR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LDMA_TX	LDMA_RX	FRXT_H	DS [3:0]				TXEIE	RXNEIE	ERRIE	FRF	NSSP	SSOE	TXDMAEN	RXDMAEN
	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

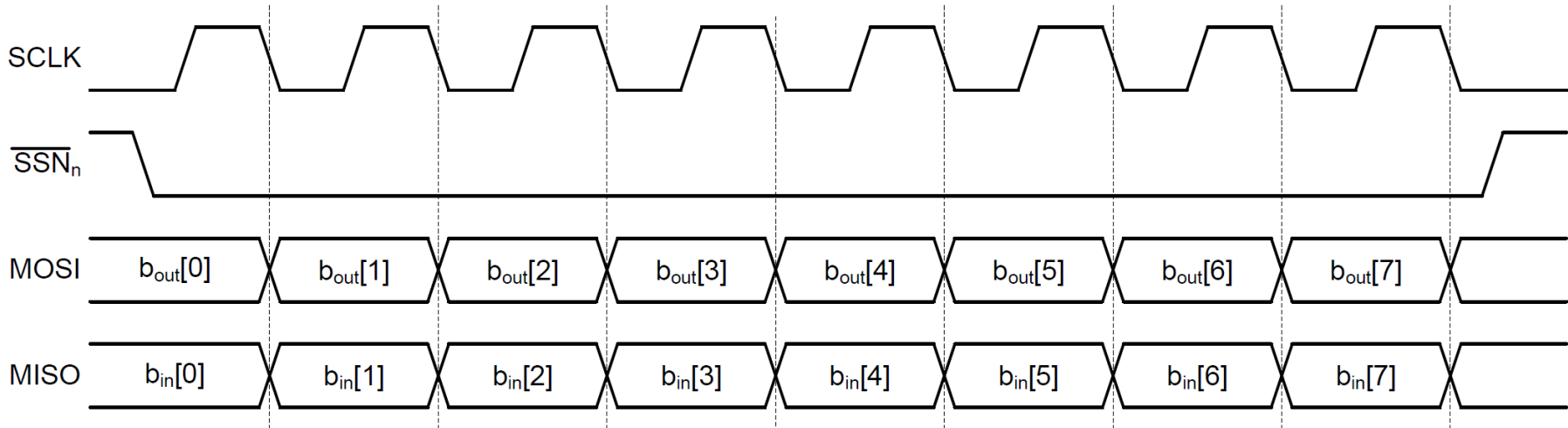
Bits 11:8 **DS [3:0]**: Data size

These bits configure the **data length for SPI transfers**:

0000: Not used
0001: Not used
0010: Not used
0011: 4-bit
0100: 5-bit
0101: 6-bit
0110: 7-bit
0111: 8-bit
1000: 9-bit
1001: 10-bit
1010: 11-bit
1011: 12-bit
1100: 13-bit
1101: 14-bit
1110: 15-bit
1111: 16-bit

If software attempts to write one of the "Not used" values, they are forced to the value "0111"(8-bit).

Clock



SPIx_CR1

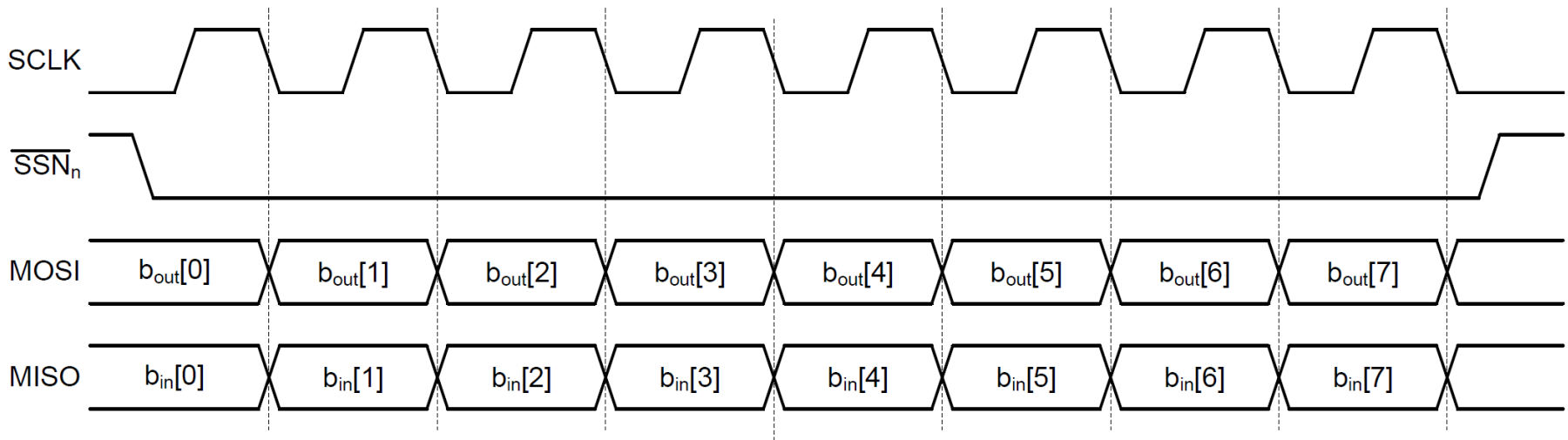
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SP	BR [2:0]			MSTR	CPOL	CPHA
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 5:3 **BR[2:0]**: Baud rate control

- 000: $f_{PCLK}/2$
- 001: $f_{PCLK}/4$
- 010: $f_{PCLK}/8$
- 011: $f_{PCLK}/16$
- 100: $f_{PCLK}/32$
- 101: $f_{PCLK}/64$
- 110: $f_{PCLK}/128$
- 111: $f_{PCLK}/256$

Note: These bits should not be changed when communication is ongoing.

Clock



SPIx_CR1

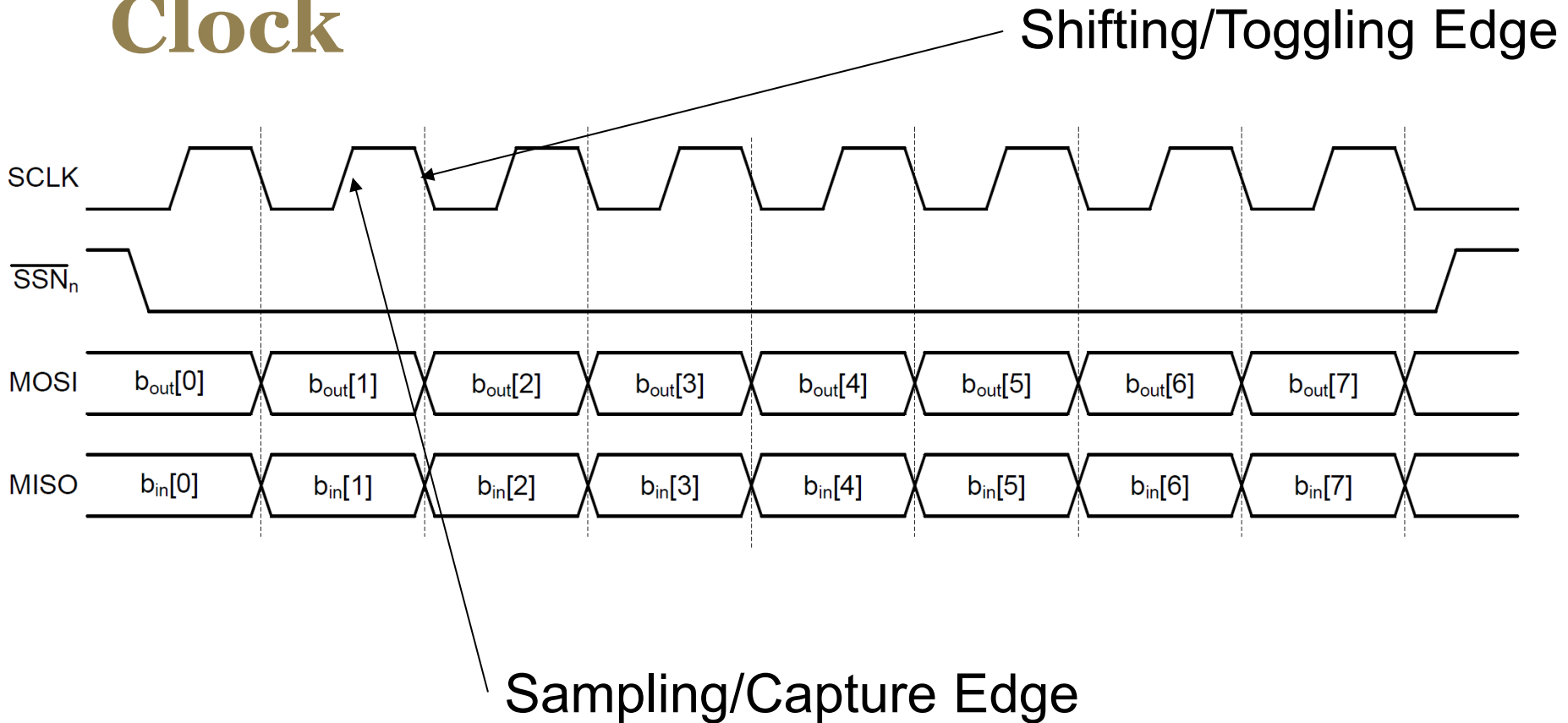
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSM	LSB FIRST	SSM	BR [2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 7 **LSBFIRST**: Frame format

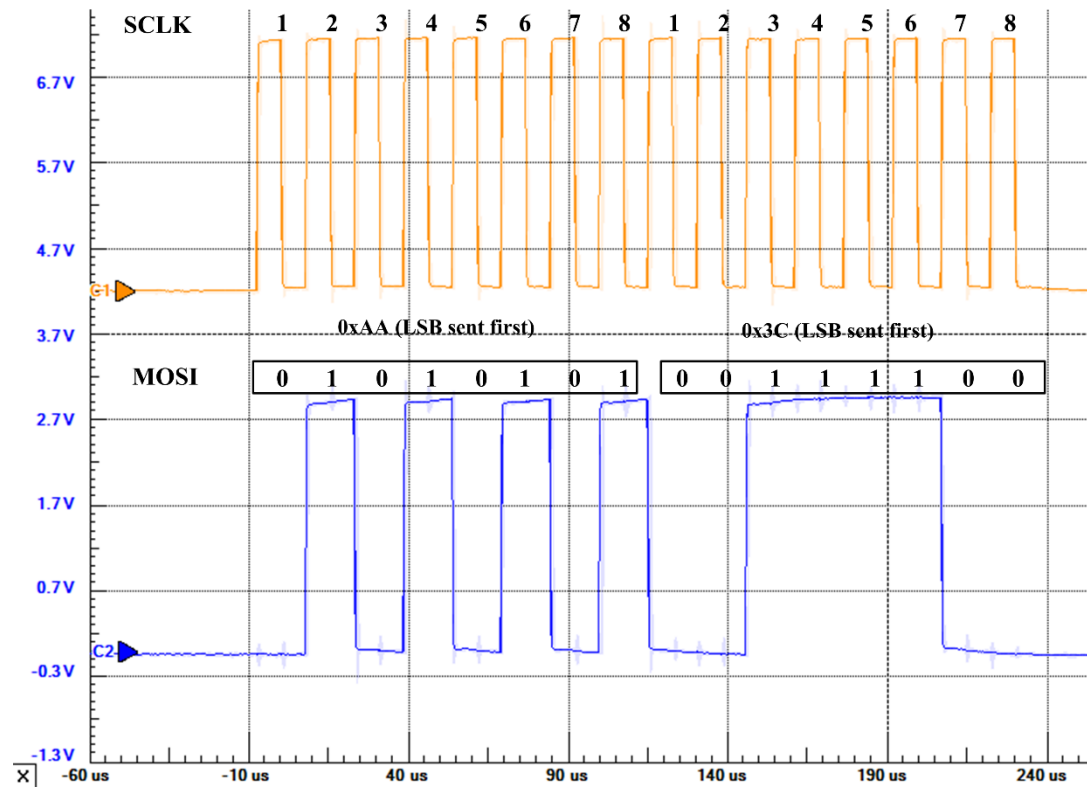
0: data is transmitted / received with the MSB first

1: data is transmitted / received with the LSB first

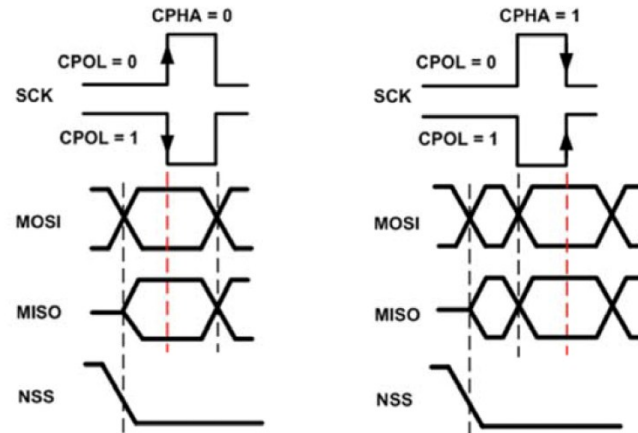
Clock



Timing Diagram - Example



Clock Modes



SPIx_CR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR [2:0]			MSR	CPOL	CPHA
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bit1 **CPOL**: Clock polarity

0: CK to 0 when idle

1: CK to 1 when idle

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode.

Bit0 **CPHA**: Clock phase

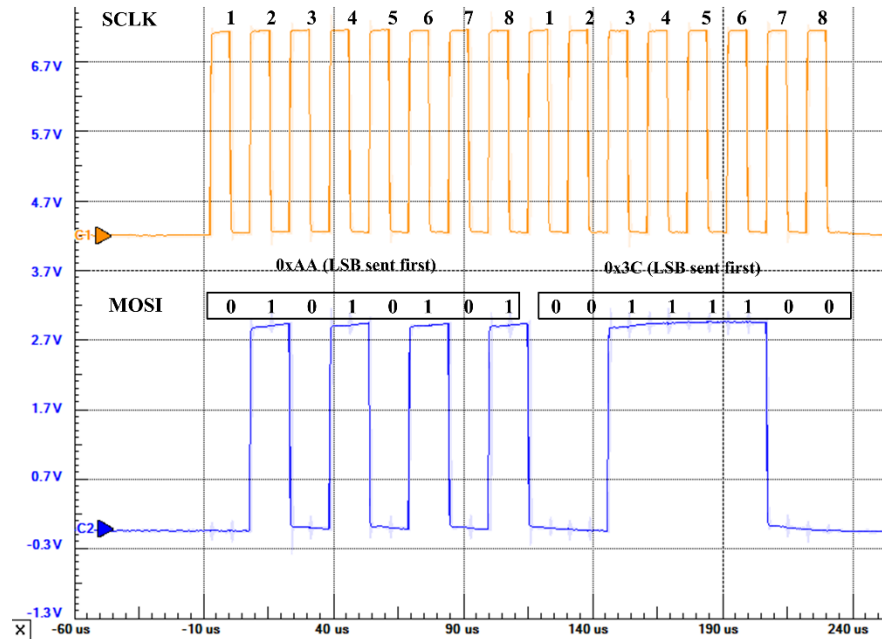
0: The first clock transition is the first data capture edge

1: The second clock transition is the first data capture edge

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode.

Timing Diagram - Example



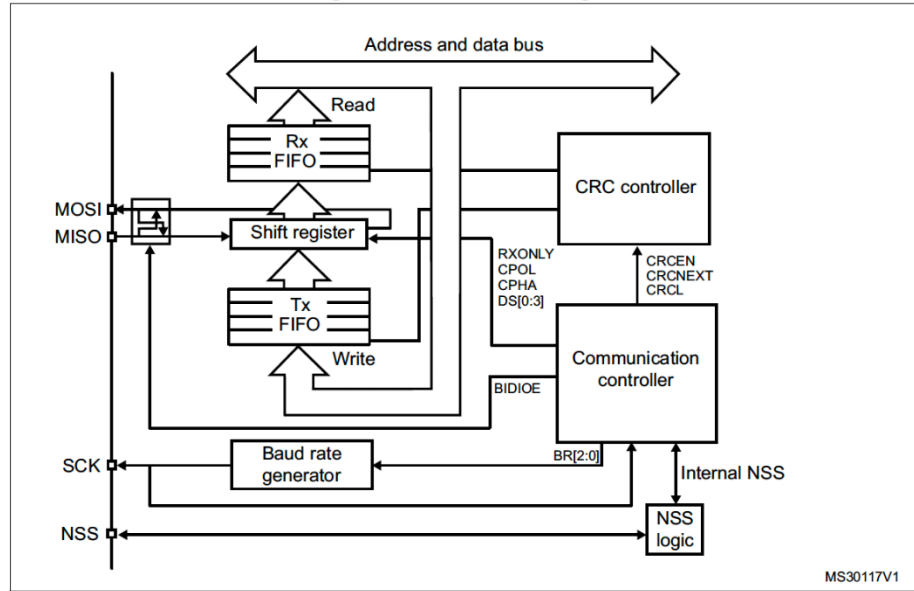
CPOL = 0, CPHA = 1, LSB First = 1



How to Program SPI

Block Diagram

Figure 416. SPI block diagram



SPIx_DR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **DR[15:0]**: Data register

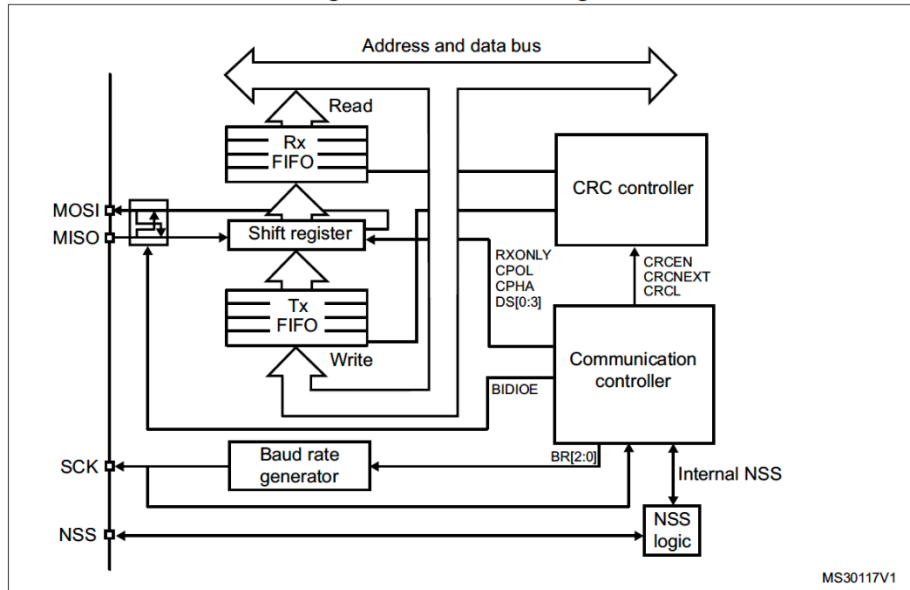
Data received or to be transmitted

The data register serves as an interface between the Rx and Tx FIFOs. When the data register is read, RxFIFO is accessed while the write to data register accesses TxFIFO (See [Section 38.4.8: Data transmission and reception procedures](#)).

Note: Data is always right-aligned. Unused bits are ignored when writing to the register, and read as zero when the register is read. The Rx threshold setting must always correspond with the read access currently used.

Block Diagram

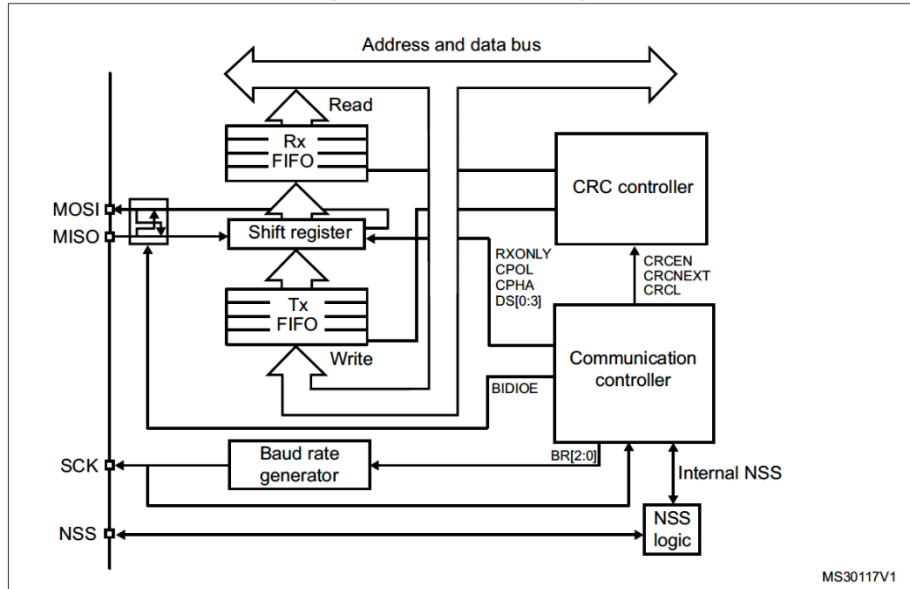
Figure 416. SPI block diagram



- A read access to the SPIx_DR register returns the oldest value stored in RXFIFO that has not been read yet.
- A write access to the SPIx_DR stores the written data in the TXFIFO at the end of a send queue.

Block Diagram

Figure 416. SPI block diagram



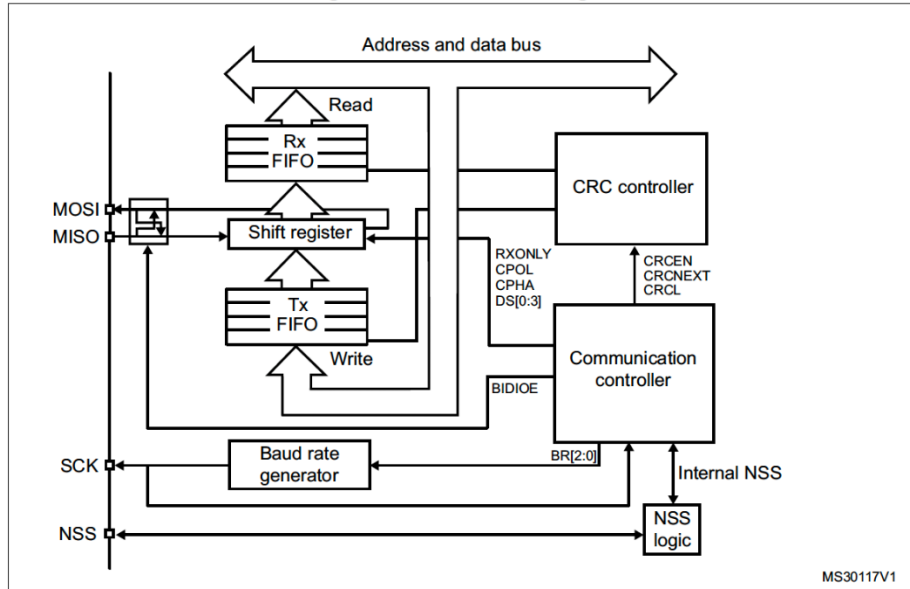
SPIx_SR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTLV[1:0]		FRLVL[2:0]		FRE	BSY	OVR	MODF	CRC ERR	Res.	Res.	TXNE	RXNE
			r	r	r	r	r	r	r	r	rc_w0			r	r

A read access to the SPIx_DR register must be managed by the RXNE event.
 FRLVL == > /* SPI fifo reception level */
 FRLVL[1:0] bits indicate the current occupancy level of RxFIFO

Block Diagram

Figure 416. SPI block diagram



SPIx_CR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LDMA_TX	LDMA_RX	FRXTH		DS [3:0]			TXEIE	RXNEIE	ERRIE	FRF	NSSP	SSOE	TXDMAEN	RXDMAEN
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 12 **FRXTH**: FIFO reception threshold

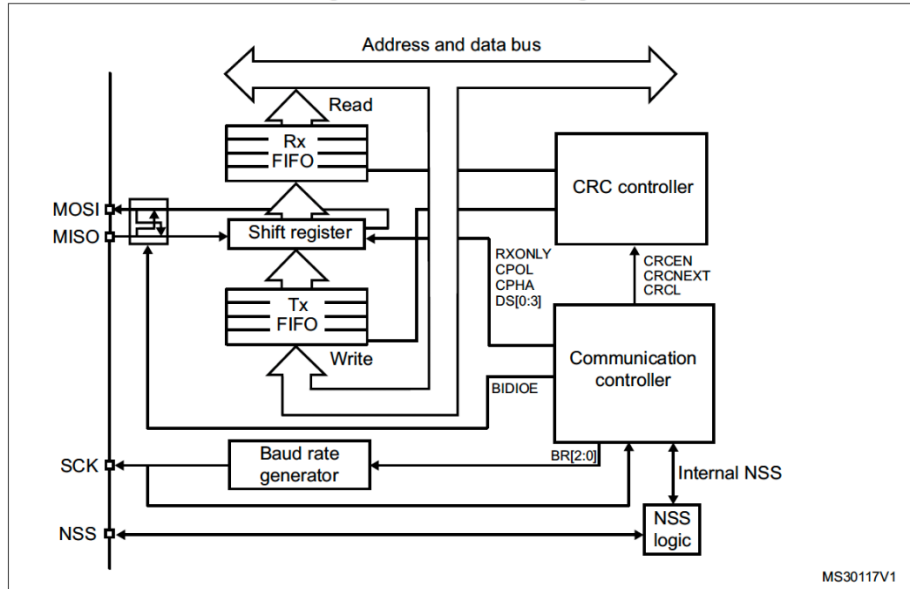
This bit is used to set the threshold of the RXFIFO that triggers an RXNE event

0: RXNE event is generated if the FIFO level is greater than or equal to 1/2 (16-bit)

1: RXNE event is generated if the FIFO level is greater than or equal to 1/4 (8-bit)

Block Diagram

Figure 416. SPI block diagram

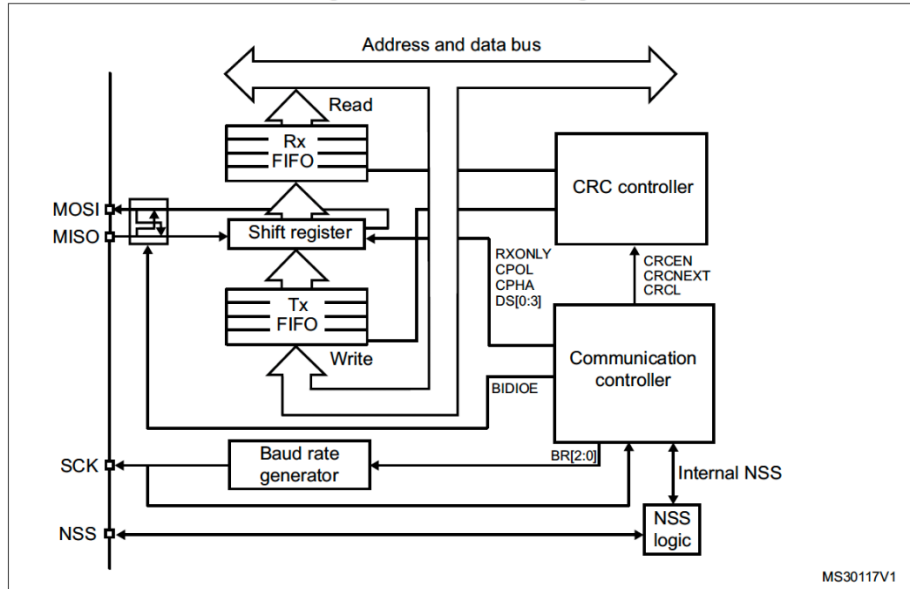


MS30117V1

- In a similar way, write access of a data frame to be transmitted is managed by the TXE event.

Block Diagram

Figure 416. SPI block diagram



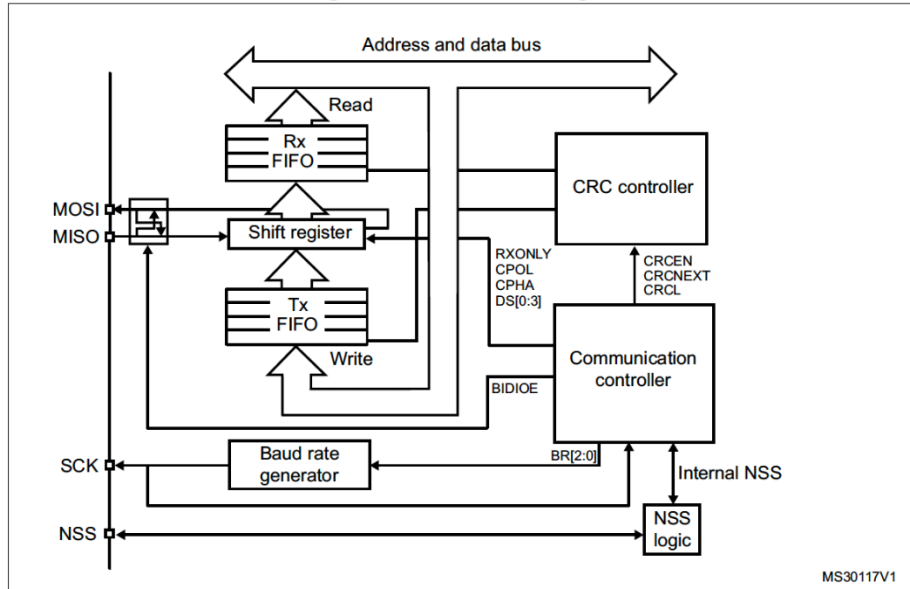
SPIx_SR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTLVL[1:0]		FRLVL[2:0]		FRE	BSY	OVR	MODF	CRC ERR	Res.	Res.	TXE	XNE
			r	r	r	r	r	r	r	r	rc_w0			r	r

- FTLVL == > /* SPI fifo transmission level */
- FTLVL[1:0] bits indicate the current occupancy level of TXFIFO
- TXE event is triggered when the TXFIFO level is less than or equal to half of its capacity.
- Otherwise TXE is cleared and the TXFIFO is considered as full.

Block Diagram

Figure 416. SPI block diagram



SPIx_SR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTLV[1:0]		FRLVL[2:0]		FRE	BSY	OR	MODF	CRC ERR	Res.	Res.	TXE	RXNE
			r	r	r	r	r	r		r	rc_w0			r	r

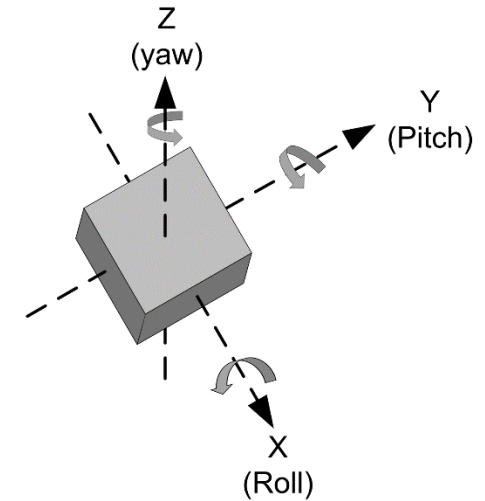
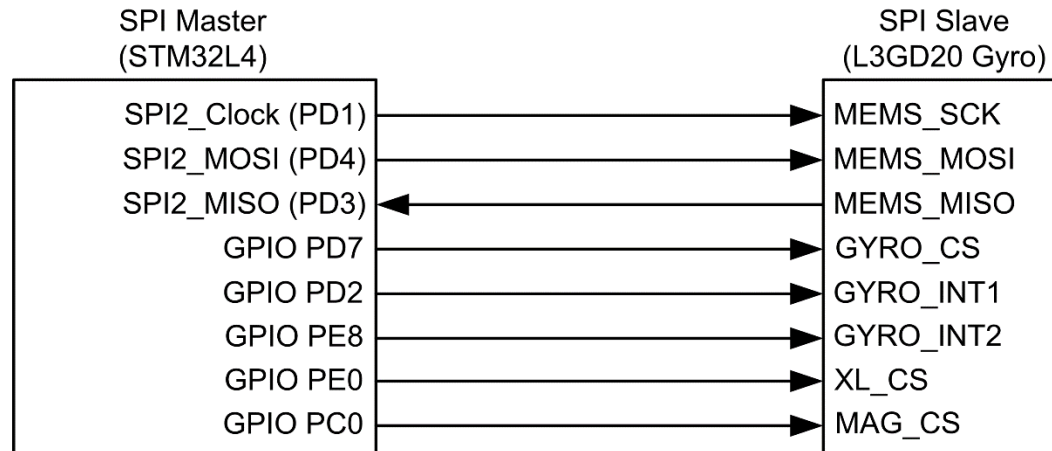
Bit 7 **BSY**: Busy flag

0: SPI not busy

1: SPI is busy in communication or Tx buffer is not empty

This flag is set and cleared by hardware.

Example





Processor's Pins

Joystick (MT-008A): Center = PA0, Left = PA1, Right = PA2, Up = PA3, Down = PA5

User LED: Red = PB2, Green = PE8

Analog Outputs: PA3 (OPAMP1_OUT), PA5 (DAC1_OUT2). Note: PA4 (DAC1_OUT1) is not extended out, thus we use OPAMP1_OUT.

Joystick (center)	PA0/TIM2_CH1/TIM5_CH1/TIM8_ETR/USART2_CTS/UART4_TX/SAI1_EXTCLK/TIM2_ETR	PC9/TIM8_BKIN/TIM3_CH4/TIM8_CH4/TSC_G4_IO4/OTG_FS_NOE/LCD_SEG27/SDMMC1_D1/SAI2_EXTCLK/TIM8_BKIN2_COMP1	OTG_FS_PowerSwitchOn
Joystick (left)	PA1/TIM2_CH2/TIM5_CH2/USART2_RTS_DE/UART4_RX/LCD_SEG0/TIM15_CH1N	PC10/SPI3_SCK/USART3_TX/UART4_TX/TSC_G3_IO2/LCD_COM4/LCD_SEG28/LCD_SEG40/SDMMC1_D2/SAI2_SCK_B	OTG_FS_OverCurrent
Joystick (right)	PA2/TIM2_CH3/TIM5_CH3/USART2_TX/LCD_SEG1/SAI2_EXTCLK/TIM15_CH1	PC11/SPI3_MISO/USART3_RX/UART4_RX/TSC_G3_IO3/LCD_COM5/LCD_SEG29/LCD_SEG41/SDMMC1_D3/SAI2_MCLK_B	OTG_FS_VBUS
Joystick (up)	PA3/TIM2_CH4/TIM5_CH4/USART2_RX/LCD_SEG2/TIM15_CH2	PC12/SPI3_MOSI/USART3_CK/UART5_TX/TSC_G3_IO4/LCD_COM6/LCD_SEG30/LCD_SEG42/SDMMC1_CK/SAI2_SD_B	OTG_FS_ID
Joystick (down)	PA4/SPI1_NSS/SPI3_NSS/USART2_CK/SAI1_FS_B/LPTIM2_OUT	PC13/PD0/SPI2_NSS/DFSDM_DATIN7/CAN1_RX/FMC_D2	OSC32_IN
LCD SEG23	PA5/TIM2_CH1/TIM2_ETR/TIM8_CH1N/SPI1_SCK/LPTIM2_ETR	PC14/	OSC32_OUT
LCD SEG0	PA6/TIM1_BKIN/TIM3_CH1/TIM8_BKIN/SPI1_MISO/USART3_CTS/QUADSPI_BK1_IO3/LCD_SEG3/TIM1_BKIN_COMP2/TIM8_BKIN_COMP2/TIM16_CH1	PC15/	
LCD COM0	PA7/TIM1_CH1N/TIM3_CH2/TIM8_CH1N/SPI1_MOSI/QUADSPI_BK1_IO2/LCD_SEG4/TIM17_CH1		
LCD COM1	PA8/MCO/TIM1_CH1/USART1_CK/OTG_FS_SOF/LCD_COM0/LPTIM2_OUT	PD1/SPI2_SCK/DFSDM_CKIN7/CAN1_TX/FMC_D3	eCompass/Gyro MEMS_SCK
LCD COM2	PA9/TIM1_CH2/USART1_TX/LCD_COM1/TIM15_BKIN	PD2/TIM3_ETR/USART3_RTS_DE/UART5_RX/TSC_SYNC/LCD_COM7/LCD_SEG31/LCD_SEG43/SDMMC1_CMD	Gyro GYRO_INT1
OTG_FS_DM	PA10/TIM1_CH3/USART1_RX/OTG_FS_ID/LCD_COM2/TIM17_BKIN	PD3/SPI2_MISO/DFSDM_DATIN0/USART2_CTS/FMC_CLK	Gyro MEMS_MISO
OTG_FS_DP	PA11/TIM1_CH4/TIM1_BKIN2/USART1_CTS/CAN1_RX/OTG_FS_DM/TIM1_BKIN2_COMP1	PD4/SPI2_MOSI/DFSDM_CKIN0/USART2_RTS_DE/FMC_NOE	eCompass/Gyro MEMS_MOSI
ST-Link SWDIO	PA12/TIM1_ETR/USART1_RTS_DE/CAN1_TX/OTG_FS_DP	PD5/USART2_TX/FMC_NWE	ST-Link USART_TX
ST-Link SWCLK	PA13/JTMS/SWDIO/IIR_OUT/OTG_FS_NOE	PD6/DFSDM_DATIN1/FMC_NWAIT/SAI1_SD_A	ST-Link USART_RX
LCD SEG10	PA14/JTCK/SWCLK	PD7/DFSDM_CKIN1/USART2_CK/FMC_NE1	Gyro GYRO_CS
	PA15/JTDI/TIM2_CH1/TIM2_ETR/SPI1_NSS/SPI3_NSS/UART4_RTS_DE/TSC_G3_IO1/LCD_SEG17/SAI2_FS_B	PD8/USART3_TX/LCD_SEG28/FMC_D13	LCD SEG18
		PD9/USART3_RX/LCD_SEG29/FMC_D14/SAI2_MCLK_A	LCD SEG5
LCD SEG21	PB0/TIM1_CH2N/TIM3_CH3/TIM8_CH2N/USART3_CK/QUADSPI_BK1_IO1/LCD_SEG5/COMP1_OUT	PD10/USART3_CK/TSC_G6_IO1/LCD_SEG30/FMC_D15/SAI2_SCK_A	LCD SEG17
LCD SEG2	PB1/TIM1_CH3N/TIM3_CH4/TIM8_CH3N/DFSDM_DATIN0/USART3_RTS_DE/QUADSPI_BK1_IO0/LCD_SEG6/LPTIM2_IN1	PD11/USART3_CTS/TSC_G6_IO2/LCD_SEG31/FMC_A16/SAI2_SD_A/LPTIM2_ETR	LCD SEG6
LED red	PB2/RTC_OUT/LPTIM1_OUT/I2C3_SMBA/DFSDM_CKIN0	PD12/TIM4_CH1/USART3_RTS_DE/TSC_G6_IO3/LCD_SEG32/FMC_A17/SAI2_FS_A/LPTIM2_IN1	LCD SEG16
ST-Link SWO	PB3/JTDO/TRACESWO/TIM2_CH2/SPI1_SCK/SPI3_SCK/USART1_RTS_DE/LCD_SEG7/SAI1_SCK_B	PD13/TIM4_CH2/TSC_G6_IO4/LCD_SEG33/FMC_A18/LPTIM2_OUT	LCD SEG7
LCD SEG11	PB4/NTRST/TIM3_CH1/SPI1_MISO/SPI3_MISO/USART1_CTS/UART5_RTS_DE/TSC_G2_IO1/LCD_SEG8/SAI1_MCLK_B/TIM17_BKIN	PD14/TIM4_CH3/LCD_SEG34/FMC_D0	LCD SEG15
LCD SEG12	PB5/LPTIM1_IN1/TIM3_CH2/I2C1_SMBB/SPI1_MOSI/SPI3_MOSI/USART1_CK/UART5_CTS/TSC_G2_IO2/LCD_SEG9/COMP2_OUT/SAI1_SD_B/TIM16_BKIN	PD15/TIM4_CH4/LCD_SEG35/FMC_D1	LCD SEG8
Codec I2C1_SCL	PB6/LPTIM1_ETR/TIM4_CH1/TIM8_BKIN/I2C1_SCL/DFSDM_DATIN5/USART1_TX/TSC_G2_IO3/TIM8_BKIN2_COMP2/SAI1_FS_B/TIM16_CH1N		
Codec I2C1_SDA	PB7/LPTIM1_IN2/TIM4_CH2/TIM8_BKIN/I2C1_SDA/DFSDM_CKIN5/USART1_RX/UART4_CTS/TSC_G2_IO4/LCD_SEG21/FMC_NL/TIM8_BKIN_COMP1/TIM17_CH1N		
Gyro GYRO_INT1	PB8/TIM4_CH3/I2C2_SCL/DFSDM_DATIN7/USART3_TX/LPUART1_RX/QUADSPI_CLK/LCD_SEG10/COMP1_OUT/SAI1_SCK_A	PE0/TIM4_ETR/LCD_SEG38/FMC_NBL0/TIM16_CH1	eCompass MAG_INT
LCD COM3	PB9/IIR_OUT/TIM4_CH4/I2C1_SDA/SPI2_NSS/DFSDM_CKIN6/CAN1_TX/LCD_COM3/SDMMC1_D5/SAI1_FS_A/TIM17_CH1	PE1/LCD_SEG37/FMC_NBL1/TIM17_CH1	eCompass XL_INT
	PB10/TIM2_CH3/I2C2_SCL/SPI2_SCK/DFSDM_DATIN7/USART3_TX/LPUART1_RX/QUADSPI_NCS/LCD_SEG11/COMP2_OUT	PE2/TRACED0/TIM3_CH1/TSC_G7_IO2/LCD_SEG39/FMC_A19/SAI1_SD_B	Codec SAH1_MCK
LCD SEG20	PB11/TIM2_CH4/I2C2_SDA/DFSDM_CKIN7/USART3_RX/LPUART1_TX/QUADSPI_NCS/LCD_SEG11/COMP2_OUT	PE3/TRACED0/TIM3_CH1/TSC_G7_IO2/LCD_SEG39/FMC_A19/SAI1_SD_B	Codec Audio_RST
LCD SEG3	PB12/TIM1_BKIN/TIM1_BKIN_COMP2/I2C2_SMBB/SPI2_NSS/DFSDM_DATIN1/USART3_CK/LPUART1_RTS_DE/TSC_G1_IO1/LCD_SEG12/SWPMI1_IO/SAI2_FS_A/TIM15_BKIN	PE4/TRACED1/TIM3_CH2/DFSDM_DATIN3/TSC_G7_IO3/FMC_A20/SAI1_FS_A	Codec SAH1_FS
LCD SEG4	PB13/TIM1_CH1N/I2C2_SCL/SPI2_SCK/DFSDM_CKIN1/USART3_CTS/LPUART1_CTS/TSC_G1_IO2/LCD_SEG13/SWPMI1_TX/SAI2_SCK_A/TIM15_CH1N	PE5/TRACED2/TIM3_CH3/DFSDM_CKIN3/TSC_G7_IO4/FMC_A21/SA1_SCK_A	Codec SAH1_SCK
	PB14/TIM1_CH2N/TIM8_CH2N/I2C2_SDA/SPI2_MISO/DFSDM_DATIN2/USART3_RTS_DE/TSC_G1_IO3/LCD_SEG14/SWPMI1_RX/SAI2_MCLK_A/TIM15_CH1	PE6/TRACED3/TIM3_CH4/FMC_A22/SAI1_SD_A	Codec SAH1_SD
	PB15/RTC_REFIN/TIM1_CH3N/TIM8_CH3N/SPI2_MOSI/DFSDM_CKIN2/TSC_G1_IO4/LCD_SEG15/SWPMI1_SUSPEND/SAI2_SD_A/TIM15_CH2	PE7/TIM1_ETR/DFSDM_DATIN2/FMC_D4/SAI1_SD_B	Audio_DIN
eCompass MAG_CS	PC0/LPTIM1_IN1/I2C3_SCL/DFSDM_DATIN4/LPUART1_RX/LCD_SEG18/LPTIM2_IN1	PE8/TIM1_CH1N/DFSDM_CKIN2/FMC_D5/SAI1_SCK_B	Audio_CLK
eCompass MAG_INT	PC1/LPTIM1_OUT/I2C3_SDA/DFSDM_CKIN4/LPUART1_TX/LCD_SEG19	PE9/TIM1_CH1/DFSDM_CKOUT/FMC_D6/SAI1_FS_B	Flash QSPI_CLK
eCompass MAG_DRDY	PC2/LPTIM1_IN2/SPI2_MISO/DFSDM_CKOUT/LCD_SEG20	PE10/TIM1_CH2N/DFSDM_DATIN4/TSC_G5_IO1/QUADSPI_CLK/FMC_D7/SAI1_MCLK_B	Flash QSPI_CS
LCD VLCD	PC3/LPTIM1_ETR/SPI2_MOSI/LCD_VLCD/SAI1_SD_A/LPTIM2_ETR	PE11/TIM1_CH2/DFSDM_CKIN4/TSC_G5_IO2/QUADSPI_NCS/FMC_D8	Flash QSPI_D0
LCD SEG22	PC4/USART3_TX/LCD_SEG22	PE12/TIM1_CH3N/SPI1_NSS/DFSDM_DATIN5/TSC_G5_IO3/QUADSPI_BK1_IO0/FMC_D9	Flash QSPI_D1
LCD SEG1	PC5/USART3_RX/LCD_SEG23	PE13/TIM1_CH3/SPI1_SCK/DFSDM_CKIN5/TSC_G5_IO4/QUADSPI_BK1_IO1/FMC_D10	Flash QSPI_D2
LCD SEG14	PC6/TIM3_CH1/TIM8_CH1/DFSDM_CKIN3/TSC_G4_IO1/LCD_SEG24/SDMMC1_D6/SAI2_MCLK_A	PE14/TIM1_CH4/TIM1_BKIN2_COMP2/SPI1_MISO/QUADSPI_BK1_IO2/FMC_D11	Flash QSPI_D3
LCD SEG9	PC7/TIM3_CH2/TIM8_CH2/DFSDM_DATIN3/TSC_G4_IO2/LCD_SEG25/SDMMC1_D7/SAI2_MCLK_B	PE15/TIM1_BKIN/TIM1_BKIN_COMP1/SPI1_MOSI/QUADSPI_BK1_IO3/FMC_D12	
LCD SEG13	PC8/TIM3_CH3/TIM8_CH3/TSC_G4_IO3/LCD_SEG26/SDMMC1_D0		OSC_IN
			OSC_OUT

STM32L476VG Discovery Kit Pin Connection

Alternate Mode

- 32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Pin 1

Pin 0

Bits $2y+1:2y$ **MODE y [1:0]**: Port x configuration bits ($y = 0..15$)

These bits are written by software to configure the I/O mode.

00: Input mode

01: General purpose output mode

10: Alternate function mode

11: Analog mode (reset state)

```
GPIOD->MODER &= ~(3UL<<2); // Clear bits 4 and 5 for Pin 2
GPIOD->MODER |= 2UL<<2; // Set bit 5, set Pin 2 as
Alternate function
```

Which Alternate Mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFSEL7[3:0]				AFSEL6[3:0]				AFSEL5[3:0]				AFSEL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFSEL3[3:0]				AFSEL2[3:0]				AFSEL1[3:0]				AFSEL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFSELy[3:0]**: Alternate function selection for port x pin y (y = 0..7)

These bits are written by software to configure alternate function I/Os

AFSELy selection:

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: AF8

1001: AF9

1010: AF10

1011: AF11

1100: AF12

1101: AF13

1110: AF14

1111: AF15

GPIOD_AFRL

Page 75 of STM32L476VGT6 Datasheet

Which Alternate Mode

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFSEL15[3:0]				AFSEL14[3:0]				AFSEL13[3:0]				AFSEL12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFSEL11[3:0]				AFSEL10[3:0]				AFSEL9[3:0]				AFSEL8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFSELY[3:0]**: Alternate function selection for port x pin y (y = 8..15)

These bits are written by software to configure alternate function I/Os

AFSELY selection:

0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15

GPIOD_AFRH

SPI Initialization

```
void SPI_Init(SPI_TypeDef * SPIx){  
  
    // Enable SPI clock  
    if(SPIx == SPI1){  
        RCC->APB2ENR   |= RCC_APB2ENR_SPI1EN;    // Enabel SPI1 Clock  
        RCC->APB2RSTR   |= RCC_APB2RSTR_SPI1RST;  // Reset SPI1  
        RCC->APB2RSTR   &= ~RCC_APB2RSTR_SPI1RST; // Clear the reset of SPI1  
    } else if(SPIx == SPI2){  
        RCC->APB1ENR1   |= RCC_APB1ENR1_SPI2EN;    // Enabel SPI2 Clock  
        RCC->APB1RSTR1  |= RCC_APB1RSTR1_SPI2RST;  // Reset SPI2  
        RCC->APB1RSTR1  &= ~RCC_APB1RSTR1_SPI2RST; // Clear reset of SPI2  
    } else if(SPIx == SPI3){  
        RCC->APB1ENR1   |= RCC_APB1ENR1_SPI3EN;    // Enabel SPI3 Clock  
        RCC->APB1RSTR1  |= RCC_APB1RSTR1_SPI3RST;  // Reset SPI3  
        RCC->APB1RSTR1  &= ~RCC_APB1RSTR1_SPI3RST; // Clear reset of SPI3  
    }  
  
    SPIx->CR1 &= ~SPI_CR1_SPE; // Disable SPI  
}
```


SPI Initialization

```
// Configure duplex or receive-only
// 0 = Full duplex (transmit and receive), 1 = Receive-only
SPIx->CR1 &= ~SPI_CR1_RXONLY;

// Bidirectional data mode enable: This bit enables half-duplex
// communication using common single bidirectional data line.
// 0 = 2-line unidirectional data mode selected
// 1 = 1-line bidirectional data mode selected
SPIx->CR1 &= ~SPI_CR1_BIDIMODE;

// Output enable in bidirectional mode
// 0 = Output disabled (receive-only mode)
// 1 = Output enabled (transmit-only mode)
SPIx->CR1 &= ~SPI_CR1_BIDIOE;

// Data Frame Format
SPIx->CR2 &= ~SPI_CR2_DS;
SPIx->CR2 = SPI_CR2_DS_0 | SPI_CR2_DS_1 | SPI_CR2_DS_2; // 0111: 8-bit
```

SPI Initialization

```
// Bit order
// 0 = MSB transmitted/received first;
// 1 = LSB transmitted/received first
SPIx->CR1 &= ~SPI_CR1_LSBFIRST; // Most significant bit first

// Clock phase
// 0 = The first clock transition is the first data capture edge
// 1 = The second clock transition is the first data capture edge
SPIx->CR1 &= ~SPI_CR1_CPHA; // 1st edge

// Clock polarity
// 0 = Set CK to 0 when idle
// 1 = Set CK to 1 when idle
SPIx->CR1 &= ~SPI_CR1_CPOL; // Polarity low

// Baud rate control:
// 000: f_PCLK/2    001: f_PCLK/4    010: f_PCLK/8    011: f_PCLK/16
// 100: f_PCLK/32   101: f_PCLK/64   110: f_PCLK/128  111: f_PCLK/256
// SPI baudrate is set to 5 MHz
SPIx->CR1 |= 3U<<3; // Set SPI clock to 80MHz/16 = 5 MHz
```

SPI Initialization

```
// Set as Master: 0 = slave, 1 = master
SPIx->CR1 |= SPI_CR1_MSTR;

// Receive buffer not empty (RXNE)
// The RXNE flag is set depending on the FRXTH bit value in the SPIx_CR2 register:
// (1) If FRXTH is set, RXNE goes high and stays high until the RXFIFO level is
//      greater or equal to 1/4 (8-bit).
// (2) If FRXTH is cleared, RXNE goes high and stays high until the RXFIFO level is
//      higher than or equal to 1/2 (16-bit).
SPIx->CR2 |= SPI_CR2_FRXTH;

// SPI enable
SPIx->CR1 |= SPI_CR1_SPE;
}
```


SPI Write

```
void SPI_Write(SPI_TypeDef * SPIx, uint8_t *txBuffer, uint8_t * rxBuffer, int size) {  
    int i = 0;  
  
    for (i = 0; i < size; i++) {  
        // Wait for TXE (Transmit buffer empty)  
        while( (SPIx->SR & SPI_SR_TXE ) != SPI_SR_TXE );  
        SPIx->DR = txBuffer[i];  
  
        // Wait for RXNE (Receive buffer not empty)  
        while( (SPIx->SR & SPI_SR_RXNE ) != SPI_SR_RXNE );  
        rxBuffer[i] = SPIx->DR;  
    }  
  
    // Wait for BSY flag cleared  
    while( (SPIx->SR & SPI_SR_BSY) == SPI_SR_BSY );  
}
```

SPI Read

```
void SPI_Read(SPI_TypeDef * SPIx, uint8_t *rxBuffer, int size) {  
    int i = 0;  
    for (i = 0; i < size; i++) {  
        // Wait for TXE (Transmit buffer empty)  
        while( (SPIx->SR & SPI_SR_TXE ) != SPI_SR_TXE );  
        // The clock is controlled by master.  
        // Thus, the master must send a byte  
        SPIx->DR = 0xFF;    // A dummy byte  
  
        // data to the slave to start the clock  
        while( (SPIx->SR & SPI_SR_RXNE ) != SPI_SR_RXNE );  
        rxBuffer[i] = SPIx->DR;  
    }  
  
    // wait for BSY flag cleared  
    while( (SPIx->SR & SPI_SR_BSY ) == SPI_SR_BSY );  
}
```



Interrupt

Polling vs Interrupt

Interrupt event	Event flag	Enable Control bit
Transmit TXFIFO ready to be loaded	TXE	TXEIE
Data received in RXFIFO	RXNE	RXNEIE

SPIx_CR2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LDMA_TX	LDMA_RX	FRXTH	DS [3:0]				TXEIE	RXNEIE	ERRIE	FRF	NSSP	SSOE	TXDMAEN	RXDMAEN
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

SPIx_SR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTLV[1:0]		FRLVL[2:0]		FRE	BSY	OVR	MODF	CRCERR	Res.	Res.	TXE	RXNE
			r	r	r	r	r	r	r	r	rc_w0			r	r

enable interrupt in NVIC using NVIC_EnableIRQ(SPIx_IRQn);

Interrupt Handler

```
void SPI1_IRQHandler() {  
    if ((SPIx->SR & SPI_SR_TXE) == SPI_SR_TXE) {  
        // Write to SPIx->DR  
    }  
    if ((SPIx->SR & SPI_SR_RXNE) == SPI_SR_RXNE) {  
        // Read from SPIx->DR  
    }  
  
    // wait for BSY flag cleared  
    while( (SPIx->SR & SPI_SR_BSY) == SPI_SR_BSY );  
}
```